

Data editor — what to build

A browser-based editor for `data/products.js`, living in the same repository as the dashboard it feeds, so that both hand over to RBM as a single artefact.

Repo: `launch-development` Branch at writing: `redesigning-illustrative-journey`

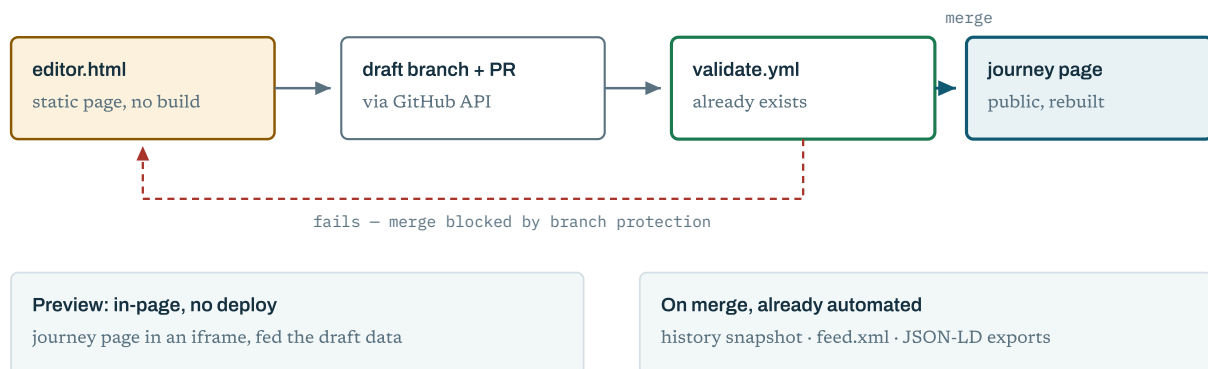
Audience: build team Status: proposed

1. The problem

Dashboard data is currently edited by hand in `data/products.js` — strict JSON after a `window.LAUNCH_DATA =` marker line. That is fine for us and unusable for the analyst at RBM who will own this after handover. They need forms, not JSON.

The obvious solutions all fail one requirement. A Next.js admin app or the existing Streamlit editor (`streamlit-app/`) both need a server process, which means a second deployment, second account and second thing for RBM to own — precisely the split we are trying to avoid. RBM's own site (`dashboards.endmalaria.org`) runs Next.js on AWS Elastic Beanstalk, but it is theirs; we cannot deploy into it, so matching their framework buys nothing.

The approach: the editor is a static page in this repository, like every other page here. It reads and writes `data/products.js` through the GitHub API using the signed-in user's own permissions. GitHub supplies storage, history, identity, access control, the validation gate and the approval mechanism — most of which this repo already uses.



The only genuinely new component is the leftmost box. Everything to its right already runs in this repo today.

2. Constraints — read before designing anything

No build step. Ever.

This repo has **no** `package.json` and no bundler. The root is served as-is, and the handover pitch to RBM is "copy the static file set anywhere". Introducing a build pipeline for the editor regresses that. Write the editor as a plain HTML page with inline or adjacent JS, exactly like `illustrated-journey-dashboard.html`. Libraries, if genuinely needed, come from a pinned CDN script tag – not npm.

One validator, shared — never a second copy

`scripts/validate-data.js` (406 lines) is the single source of truth for what valid data is, and it has a SHACL twin in `ontology/launch-shapes.ttl` that must change alongside it. The editor must run *the same rules* in the browser. A hand-written JS copy of the rules will drift, and the moment the two disagree nobody can trust either.

Only `products.js` is editable

`data/resistance.js` and `data/molecular-markers.js` carry GENERATED ... Do not edit by hand headers and are reproduced byte-identically by `scripts/normalize-resistance.js` and `scripts/normalize-molecular-markers.js` from `sourcing/raw/`. Editing them through a form would silently break that guarantee. **Do not surface them in the editor at all** – not even read-only, where a future contributor might add a save button.

The editor must never enter the public bundle

`scripts/build-public-site.sh` copies an explicit allowlist of files into `public-site/`. Safe by default: leaving the editor off that list is all the privacy enforcement required. Add a comment in that script saying so, because the omission looks like an oversight to anyone who does not know.

Line endings and NUL bytes

Working copies are CRLF, the repo stores LF. The serializer must write `\n` (the Python one already does, via `newline="\n"`). Review diffs with `git diff --ignore-cr-at-eol`. Separately: check output for NUL bytes before committing – a NUL is a legal string character, so the page and tests pass while git reclassifies the file as binary and the diff becomes unreviewable. `grep -P '\x00'` has produced false negatives here; count bytes in Python instead.

3. Build items, in dependency order

1 · Extract the validation rules into a shared module **BLOCKING**

Today `scripts/validate-data.js` is a top-level script: it reads `process.argv`, pushes into module-level `errors` / `warnings` arrays and exits. Nothing can import it.

Refactor to a pure function with no I/O and no globals, in a file that loads in both environments – CommonJS export for Node, a global assignment for the browser:

```
// scripts/data-rules.js
function checkData(data) { /* ... */ return { errors, warnings }; }
```

`scripts/validate-data.js` becomes a thin CLI wrapper: read the file, extract the JSON body, call `checkData`, print, set the exit code. **Behaviour must not change.**

ACCEPTANCE

`node scripts/validate-data.js` → **0 errors, 5 warnings** (3 resistance: + 2 molecular markers:).

`node scripts/validate-data.js data/products.synthetic.js` → **0 errors, 0 warnings**.

Any other split means the refactor changed behaviour. Also rerun both normalizers and confirm the generated files come back byte-identical.

2 · Port the house-style serializer to JS **PORT**

Saving must reproduce the repo's exact formatting, or every save produces a whole-file diff and review becomes impossible. A working implementation already exists in Python at `streamlit-app/launch_data.py`: `_fmt()` (recursive formatter, with a special case for depth inside `products`), `file_header()` (preserves the existing comment header up to and including the `window.LAUNCH_DATA = marker`) and `serialize_products_js()`.

ACCEPTANCE

Round-trip: parse `data/products.js`, serialize with no edits, compare to the original. Must be byte-identical. Same for `data/products.synthetic.js`.

3 · `editor.html` — the editing surface NEW

The largest item, but not the hardest: `streamlit-app/edit_tab.py` (402 lines) has already worked out every field, editor grid and save gate. Treat it as the functional spec and rebuild the packaging.

Scope, following the analyst guide's update loop:

- Stage statuses, notes, dates, next steps, sources and `asOf`
- Bottleneck flag, current stage, poster phase, price, country counts
- Milestones, journey gates, the country-map list
- Add / delete / placeholder products; `meta.dataStatus`
- A raw-JSON escape hatch for everything else (volume, research) – keeps the editor from becoming a blocker on schema growth

Validation runs live against the working draft via `checkData`. Save is gated exactly as a manual edit is: a changelog description is required, added newest-first, with `meta.lastUpdated` bumped to today. Note the validator warns if changelog dates are not newest-first, because `feed.xml` takes the first N entries as newest.

ACCEPTANCE

An analyst can complete the full update loop – change a stage status, add a source, write a changelog line, save – without seeing JSON. Saving a draft with a missing source is refused, with the failing rule named on screen.

4 · In-page preview NEW

The analyst must see the change on the real page before publishing. Do **not** build this as a deploy-per-branch: that couples us to a specific host, and the hosting decision belongs to RBM.

Instead render locally – load `illustrated-journey-dashboard.html` into an iframe from a blob URL with the draft data substituted for the `data/products.js` script tag. The page reads `window.LAUNCH_DATA` plus `LAUNCH_MAP`, `LAUNCH_MAP_GEO`, `LAUNCH_RESISTANCE` and `LAUNCH_MOLECULAR_MARKERS`; only the first is being edited, so the rest load unchanged.

This is faster than any deploy-based preview – it updates as they type – and removes a hosting dependency.

5 · GitHub integration NEW

Four operations: read the current `data/products.js`; create a branch; commit the serialized file to it; open a pull request. Publishing merges that PR. The analyst never sees the words "branch" or "pull request" – the buttons say **Save draft** and **Publish**.

All calls use the signed-in user's own token, so GitHub's permissions are the authorisation model. The page holds no credential of its own and no secrets are built into it.

Consequence worth stating plainly: even a fully compromised editor page can only open a pull request. It cannot publish. That property is why this design needs no auth system of its own – do not undermine it by adding a service token with write access to `main`.

6 · Sign-in — decision required DECIDE

Two viable options. The answer depends on whether RBM's IT will permit staff GitHub accounts, which is still unconfirmed.

OPTION	MECHANISM	COST	ATTRIBUTION
A – Fine-grained PAT	Analyst creates a token scoped to this repo, pastes it once, held in <code>sessionStorage</code>	None. Truly zero server.	Real, proven by GitHub
B – Sign in with GitHub	OAuth; needs one small serverless function (~40 lines) for the code-for-token exchange, since the secret cannot ship in a static page	One function, same repo	Real, proven by GitHub

Recommendation: B. The analyst is another organisation's staff member with no one to ask when they get stuck; a one-time token-creation flow is a poor thing to hand them. Forty lines is a fair price. If the OAuth app cannot be created, A works and the guide needs screenshots.

Fallback if GitHub accounts are refused entirely: an access gate in front of the editor keyed to RBM work email, with writes made by a single machine account and the real person's name and email recorded in the changelog entry. Attribution then rests on our editor rather than on GitHub – weaker, and worth stating as such, but acceptable. Costs more to build than either A or B.

7 · Repository configuration CONFIG

No code. Set on the repo:

- Branch protection on `main`, with the `validate` check required – this is what makes "cannot publish invalid data" true rather than merely intended. `validate.yml` already runs on `pull_request`.
- Two teams: editors (open PRs) and approvers (merge).
- Confirm `publish.yml` still behaves when the data change arrives by merge rather than direct push. It is path-filtered on `data/products.js` and pushes as `launch-dashboard-bot`; the append-only history guard will fail the run if `meta.lastUpdated` was not bumped – which the editor always does, but verify it end to end.

8 · Analyst documentation NEW

Extend `docs/data-analyst-guide.md` with the editor loop. Written for someone who has never seen the repo and has nobody to ask. Per `CLAUDE.md`, this lands in the same commit as the code it describes.

4. What already exists — do not rebuild

CAPABILITY	WHERE
Data validation, incl. SHACL governance twin	scripts/validate-data.js · validate.yml
Append-only history snapshots	publish.yml → history/
RSS feed rebuild	scripts/make-feed.js
JSON-LD dataset + temporal exports	scripts/build-ontology.js · build-history-graph.js
Public/private file separation	scripts/build-public-site.sh
Every editing form and save gate, in Python	streamlit-app/edit_tab.py
House-style serializer, in Python	streamlit-app/launch_data.py

The Streamlit app stays as the internal analyst workbench – flexible data sources, what-if configuration, pre-flight review of draft files. It is not part of the handover artefact, because it needs a Python host.

5. Verify block before any commit

```
node scripts/normalize-resistance.js           # regenerate; expect byte-identical
node scripts/normalize-molecular-markers.js    # ditto
node scripts/validate-data.js                  # expect 0 errors, 5 warnings
node scripts/validate-data.js data/products.synthetic.js  # expect 0 errors, 0
warnings
node scripts/make-preview.js                   # smoke test
```

The 5 warnings are 3 + 2: three long-standing `resistance`: (1 unnamed site, 730 uncited studies, 12 undrawn country values) and two `molecular markers`: (625 uncited surveys, 9 undrawn country values). Any other split means something moved. `scripts/verify-map-clusters.js` needs puppeteer, which is not installed.

Two failure modes that have already happened here, repeated because they cost real time: check for NUL bytes before committing, and when a test disagrees with the code, establish which is wrong before changing either – on this branch the test was wrong both times.

6. Open questions blocking full scope

QUESTION	BLOCKS	ASK
Can RBM staff hold GitHub accounts?	Item 6, and the fallback's cost	RBM IT
Will endmalaria.org embed the journey page, or copy the files?	Whether publishing stays automatic after handover	RBM web team
Who owns the repo and hosting post-handover?	Where the OAuth app and hosting are created – cheaper to get right first time	RBM
Is endmalaria.org run in-house or by a contractor?	Who we are actually handing over to	RBM

None of these block items 1–5. Start with item 1: it is the dependency for everything else, it is self-contained, and its acceptance test is unambiguous.